hashlock.

Security Audit

Jedai (AI)

February 2025 info@hashlock.com.au Hashlock Pty Ltd

Table of Contents

CAUTION

THIS DOCUMENT IS A SECURITY AUDIT REPORT AND MAY CONTAIN CONFIDENTIAL INFORMATION. THIS INCLUDES IDENTIFIED VULNERABILITIES AND MALICIOUS CODE WHICH COULD BE USED TO COMPROMISE THE PROJECT. THIS DOCUMENT SHOULD ONLY BE FOR INTERNAL USE UNTIL ISSUES ARE RESOLVED. ONCE VULNERABILITIES ARE REMEDIATED, THIS REPORT CAN BE MADE PUBLIC. THE CONTENT OF THIS REPORT IS OWNED BY HASHLOCK PTY LTD FOR USE OF THE CLIENT.

Executive Summary

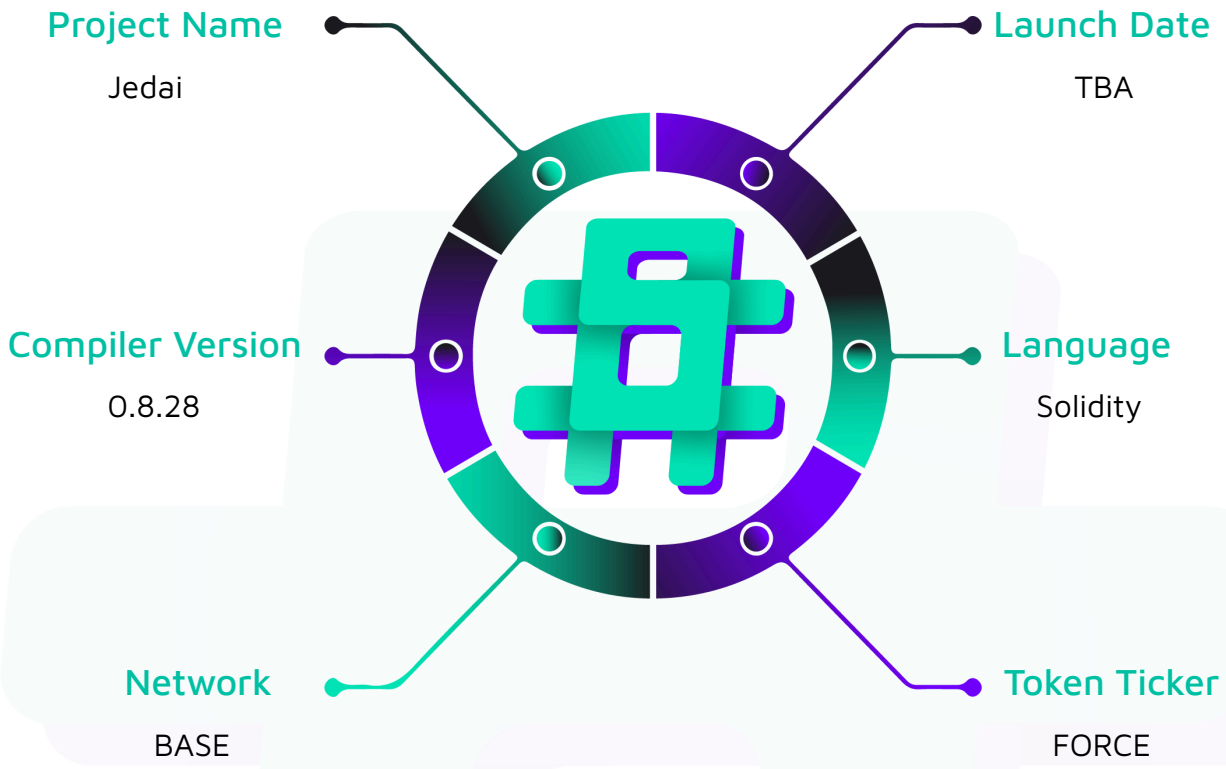
The Jedai team partnered with Hashlock to conduct a security audit of their JedaiForce.sol smart contract. Hashlock manually and proactively reviewed the code in order to ensure the project's team and community that the deployed contracts are secure.

Project Context

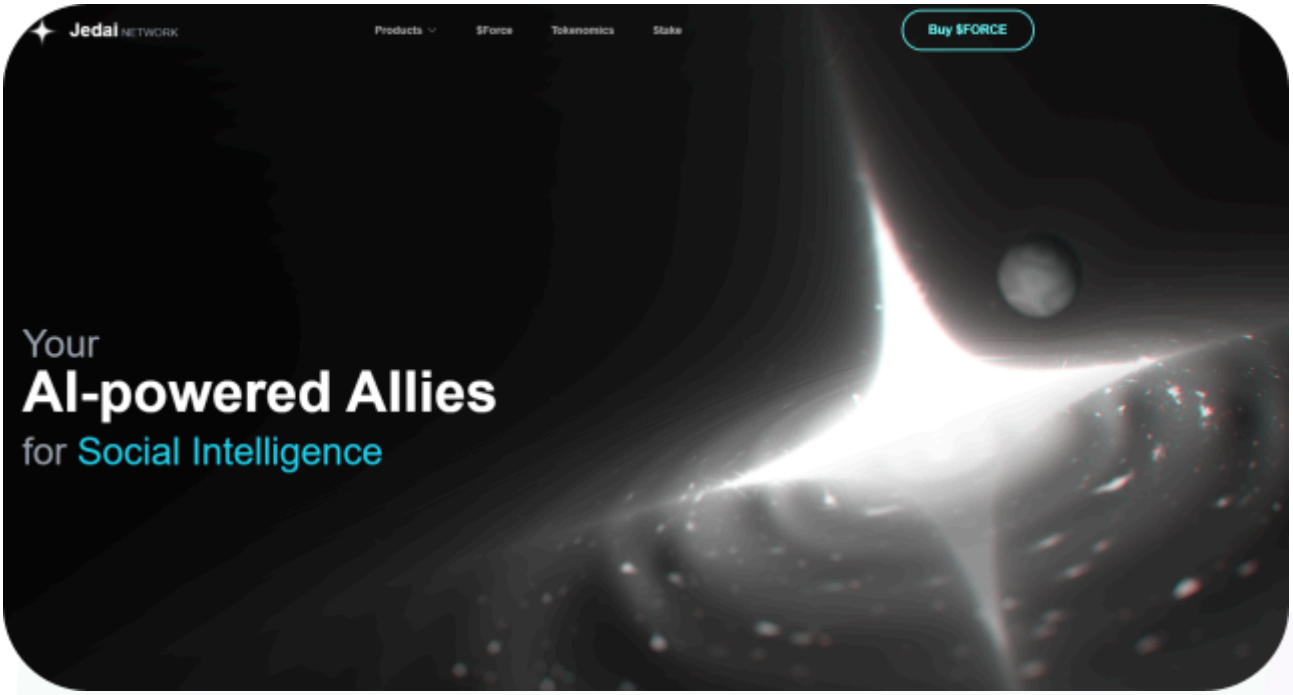
Jedai Network is a cutting-edge Web3 platform that leverages AI-powered agents to transform social and market data into actionable intelligence for the crypto and broader digital ecosystem. Through its suite of products—Jedai One, Two, and the pioneering Jedai Zero—users can monitor market sentiment, receive real-time alerts on NFT mints, whitelists, and airdrops, and gain exclusive insights to drive informed decisions. The ecosystem is powered by the $FORCE token, which provides premium access, staking benefits, and unique rewards such as the Jedai Genesis NFT for top stakers. This integration of AI, social intelligence, and blockchain technology positions Jedai Network as a gateway to next-generation digital transformation and market intelligence.

Project Name: Jedai
Compiler Version: 0.8.28
Website: https://www.jedai.network/
Logo:

Visualised Context:



Project Visuals:



Audit Scope

We at Hashlock audited the solidity code within the Jedai project, the scope of work included a comprehensive review of the smart contracts listed below. We tested the smart contracts to check for their security and efficiency. These tests were undertaken primarily through manual line-by-line analysis and were supported by software-assisted testing.

Description	**Jedai Protocol Smart Contracts**
Platform	**Base / Solidity**
Audit Date	**Feburary, 2025**
Contract 1	JedaiForce.sol
Contract 1 MD5 Hash	47170ed31b8858afb388eb4bd074bd1f
GitHub Commit Hash	188fddcfa20537afe3b791a59ebee104326ccdfb

Security Rating

After Hashlock's Audit, we found the smart contracts to be **"Secure"**. The contracts all follow simple logic, with correct and detailed ordering. They use a series of interfaces, and the protocol uses a list of Open Zeppelin contracts. We initially identified some vulnerabilities that have since been addressed.

The 'Hashlocked' rating is reserved for projects that ensure ongoing security via bug bounty programs or on chain monitoring technology.

All issues uncovered during automated and manual analysis were meticulously reviewed and applicable vulnerabilities are presented in the Audit Findings section. The general security overview is presented in the Standardised Checks section and the project's contract functionality is presented in the Intended Smart Contract Functions section.

All vulnerabilities initially identified have now been resolved and acknowledged.

Hashlock found:

2 Low severity vulnerabilities

2 Gas Optimizations

2 QAs

Caution: *Hashlock's audits do not guarantee a project's success or ethics, and are not liable or responsible for security. Always conduct independent research about any project before interacting.*

Intended Smart Contract Functions

Claimed Behaviour	Actual Behaviour
JedaiForce.sol • Allows users to: - Claim tokens if they are in merkle tree - Claim partial amounts from their allocation - Track their claimed amounts • Allows admins to: - Mint tokens to addresses - Set claimable supply - Set merkle root - Upgrade the contract	**Contract achieves this functionality.**

Code Quality

This audit scope involves the smart contracts of the Jedai project, as outlined in the Audit Scope section. All contracts, libraries, and interfaces mostly follow standard best practices and to help avoid unnecessary complexity that increases the likelihood of exploitation, however, some refactoring was recommended.

The code is very well commented on and closely follows best practice nat-spec styling. All comments are correctly aligned with code functionality.

Audit Resources

We were given the Jedai project smart contract code in the form of Github access.

As mentioned above, code parts are well commented. The logic is straightforward, and therefore it is easy to quickly comprehend the programming flow as well as the complex code logic. The comments are helpful in providing an understanding of the protocol's overall architecture.

Dependencies

As per our observation, the libraries used in this smart contracts infrastructure are based on well-known industry standard open source projects.
Apart from libraries, its functions are used in external smart contract calls.

Severity Definitions

The severity levels assigned to findings represent a comprehensive evaluation of both their potential impact and the likelihood of occurrence within the system. These categorizations are established based on Hashlock's professional standards and expertise, incorporating both industry best practices and our discretion as security auditors. This ensures a tailored assessment that reflects the specific context and risk profile of each finding.

Significance	Description
High	High-severity vulnerabilities can result in loss of funds, asset loss, access denial, and other critical issues that will result in the direct loss of funds and control by the owners and community.
Medium	Medium-level difficulties should be solved before deployment, but won't result in loss of funds.
Low	Low-level vulnerabilities are areas that lack best practices that may cause small complications in the future.
Gas	Gas Optimisations, issues, and inefficiencies
QA	Quality Assurance (QA) findings are informational and don't impact functionality. Supports clients improve the clarity, maintainability, or overall structure of the code.

Status Definitions

Each identified security finding is assigned a status that reflects its current stage of remediation or acknowledgment. The status provides clarity on the handling of the issue and ensures transparency in the auditing process. The statuses are as follows:

Significance	Description
Resolved	The identified vulnerability has been fully mitigated either through the implementation of the recommended solution proposed by Hashlock or through an alternative client-provided solution that demonstrably addresses the issue
Acknowledged	The client has formally recognized the vulnerability but has chosen not to address it due to the high cost or complexity of remediation. This status is acceptable for medium and low-severity findings after internal review and agreement. However, all high-severity findings must be resolved without exception.
Unresolved	The finding remains neither remediated nor formally acknowledged by the client, leaving the vulnerability unaddressed.

Audit Findings

Low

[L-01] JedaiForce#setClaimableSupply - Insufficient Validation for `_claimableSupply` can lead to transaction failure.

Description

The `_claimableSupply` variable determines the number of tokens available for users to claim. However, the `setClaimableSupply` function lacks a validation mechanism to ensure that the configured `_claimableSupply` does not exceed the remaining token supply `(MAX_SUPPLY - totalSupply())`. If `_claimableSupply` is set incorrectly, it could lead to an attempt to mint more tokens than allowed by the cap, causing transactions to fail.

Recommendation

Add this respective safety check to the function

```
function setClaimableSupply(uint256 amount) public onlyOwner {

    require(totalSupply() + amount <= cap(), "Claimable supply exceeds cap");

    _claimableSupply = amount;

}
```

Status

Resolved

[L-02] JedaiForce - Use `Ownable2StepUpgradeable` version rather than `OwnableUpgradeable` version

Description

`Ownable2StepUpgradeable` and `Ownable2Step` prevent the contract ownership from mistakenly being transferred to an address that cannot handle it (e.g. due to a typo in the address), by requiring that the recipient of the owner permissions actively accept via a contract call of its own.

Recommendation

Consider using `Ownable2StepUpgradeable` and `Ownable2Step` from OpenZeppelin Contracts to enhance the security of your contract ownership management. This contract prevents the accidental transfer of ownership to an address that cannot handle it, such as due to a typo, by requiring the recipient of owner permissions to actively accept ownership via a contract call.

Status

Resolved

Gas

[G-01] JedaiForce#_update - Redundant Cap Enforcement in `_update` function

Description

The `_update` function overrides the parent implementation to enforce the token supply cap by checking whether the total supply exceeds the maximum cap (`MAX_SUPPLY`) during minting operations:

```
if (from == address(0)) {

    ...

    if (supply > maxSupply) {

        revert ERC20ExceededCap(supply, maxSupply);

    }

}
```

However, this check is redundant because the `ERC20CappedUpgradeable` contract already enforces the cap within its `_mintfunction`. The additional validation in `_update` does not provide any extra security and unnecessarily increases gas costs for every minting operation.

Recommendation

Remove the logic checks

```
function _update(address from, address to, uint256 value) internal virtual override(ERC20CappedUpgradeable, ERC20Upgradeable) {

    super._update(from, to, value);

}
```

Status

Resolved

[G-02] JedaiForce#claimFor - State variable that is used multiple times in a function should be cached

Description

The `claimFor` function reads the `stakingContractAddress` value from storage multiple times.

When performing multiple operations on a state variable in a function, it is recommended to cache it first. Either multiple reads or multiple writes to a state variable can save gas by caching it on the stack.

Recommendation

Cache the `stakingContractAddress` value into a memory and use it.

Status

Resolved

QA

[Q-01] JedaiForce - Missing event emissions in functions

Description

The following functions do not emit events to log critical state changes or actions.

```
function setClaimableSupply() // JedaiForce.sol
function setMerkleRoot() // JedaiForce.sol
function setStakingContractAddress() // JedaiForce.sol
```

Recommendation

It is recommended to add relevant events to the functions.

Status

Resolved

[Q-02] JedaiForce - Floating pragma

Description

The contract uses `pragma solidity ^0.8.19`.

This can allow the contracts to be deployed with the wrong pragma version which is different from the one the contracts were tested with.

Recommendation

Lock the pragma version.

Status

Resolved

Centralisation

The Jedai project values security and utility over decentralisation.

The owner executable functions within the protocol increase security and functionality but depend highly on internal team responsibility.

Conclusion

After Hashlock's analysis, the Jedai project seems to have a sound and well-tested code base, now that our vulnerability findings have been resolved and acknowledged. Overall, most of the code is correctly ordered and follows industry best practices. The code is well commented on as well. To the best of our ability, Hashlock is not able to identify any further vulnerabilities.

Our Methodology

Hashlock strives to maintain a transparent working process and to make our audits a collaborative effort. The objective of our security audits is to improve the quality of systems and upcoming projects we review and to aim for sufficient remediation to help protect users and project leaders. Below is the methodology we use in our security audit process.

Manual Code Review:

In manually analysing all of the code, we seek to find any potential issues with code logic, error handling, protocol and header parsing, cryptographic errors, and random number generators. We also watch for areas where more defensive programming could reduce the risk of future mistakes and speed up future audits. Although our primary focus is on the in-scope code, we examine dependency code and behaviour when it is relevant to a particular line of investigation.

Vulnerability Analysis:

Our methodologies include manual code analysis, user interface interaction, and white box penetration testing. We consider the project's website, specifications, and whitepaper (if available) to attain a high-level understanding of what functionality the smart contract under review contains. We then communicate with the developers and founders to gain insight into their vision for the project. We install and deploy the relevant software, exploring the user interactions and roles. While we do this, we brainstorm threat models and attack surfaces. We read design documentation, review other audit results, search for similar projects, examine source code dependencies, skim open issue tickets, and generally investigate details other than the implementation.

Documenting Results:

We undergo a robust, transparent process for analysing potential security vulnerabilities and seeing them through to successful remediation. When a potential issue is discovered, we immediately create an issue entry for it in this document, even though we have not yet verified the feasibility and impact of the issue. This process is vast because we document our suspicions early even if they are later shown to not represent exploitable vulnerabilities. We generally follow a process of first documenting the suspicion with unresolved questions, and then confirming the issue through code analysis, live experimentation, or automated tests. Code analysis is the most tentative, and we strive to provide test code, log captures, or screenshots demonstrating our confirmation. After this, we analyse the feasibility of an attack in a live system.

Suggested Solutions:

We search for immediate mitigations that live deployments can take and finally, we suggest the requirements for remediation engineering for future releases. The mitigation and remediation recommendations should be scrutinised by the developers and deployment engineers, and successful mitigation and remediation is an ongoing collaborative process after we deliver our report, and before the contract details are made public.

Disclaimers

Hashlock's Disclaimer

Hashlock's team has analysed these smart contracts in accordance with the best industry practices at the date of this report, in relation to: cybersecurity vulnerabilities and issues in the smart contract source code, the details of which are disclosed in this report, (Source Code); the Source Code compilation, deployment, and functionality (performing the intended functions).

Due to the fact that the total number of test cases is unlimited, the audit makes no statements or warranties on the security of the code. It also cannot be considered as a sufficient assessment regarding the utility and safety of the code, bug-free status, or any other statements of the contract. While we have done our best in conducting the analysis and producing this report, it is important to note that you should not rely on this report only. We also suggest conducting a bug bounty program to confirm the high level of security of this smart contract.

Hashlock is not responsible for the safety of any funds and is not in any way liable for the security of the project.

Technical Disclaimer

Smart contracts are deployed and executed on a blockchain platform. The platform, its programming language, and other software related to the smart contract can have their own vulnerabilities that can lead to attacks. Thus, the audit can't guarantee the explicit security of the audited smart contracts.

About Hashlock

Hashlock is an Australian-based company aiming to help facilitate the successful widespread adoption of distributed ledger technology. Our key services all have a focus on security, as well as projects that focus on streamlined adoption in the business sector.

Hashlock is excited to continue to grow its partnerships with developers and other web3-oriented companies to collaborate on secure innovation, helping businesses and decentralised entities alike.

Website: hashlock.com.au
Contact: info@hashlock.com.au